GAPLYTICS

AI-Adaptive Onboarding Engine

Close the Gap. Own the Role.

Version 1.0 • FastAPI + React + AWS Bedrock

1. Project Overview

Gaplytics is a dual-role AI-powered onboarding engine that bridges the gap between what candidates know and what roles require. It combines PDF resume extraction, AI skill analysis, graph-based learning pathway generation, and score-gated mock technical interviews into a single cohesive platform.

The platform is designed for two primary users:

- HR Managers / Hiring Teams — post job descriptions, generate shareable candidate links, view the full candidate pipeline with match scores, compare candidates side-by-side using AI verdicts, and track team readiness in real time.
- New Hires / Candidates — upload a PDF resume, receive a precise skill gap analysis against the job description, follow a day-by-day personalized learning roadmap, chat with an AI coach, and unlock a 12-question mock technical interview upon achieving a 90%+ match score.

> **Tagline**
> "Close the Gap. Own the Role." — The only onboarding platform that builds a personalized training path for every hire, from day one to full competency.

> **Live Deployment**
> Frontend: https://gaplytics.vercel.app
> Backend: https://gaplytics-backend.onrender.com (AWS Bedrock — Claude 3 Haiku)
> Repository: https://github.com/balapraharsha/Gaplytics

2. Architecture & System Design

The system is split into two independently deployed services connected via REST API. All AI calls route through AWS Bedrock using Bearer token authentication — no AWS SDK or boto3 dependency is required.

2.1 High-Level Data Flow

```
HR Portal                                  Candidate Portal
──────────────────────────────             ──────────────────────────────────────
1. POST /api/hr/create-role   →            1. Upload resume PDF
   JD text / PDF uploaded                   2. POST /api/analyze
   Claude extracts requirements             3. PyMuPDF → raw text extraction
   role_id UUID generated                   4. Claude → skill array (JSON)
2. GET /api/hr/role/{id}                    5. Weighted gap scoring
   Candidate pipeline view                  6. Kahn topological sort (DAG)
3. POST /api/hr/compare                     7. Deadline bin-packing scheduler
   AI-generated verdict                     8. Score gate: ≥90% → interview
```

2.2 Service Architecture

Layer	Technology	Purpose
Frontend	React 18 + Vite + TailwindCSS	SPA — all candidate and HR UI
Backend	FastAPI + Python 3.11 + Uvicorn	REST API — all business logic
AI / LLM	AWS Bedrock (Claude 3 Haiku)	Extraction, coaching, interview gen/eval
PDF Parsing	PyMuPDF (fitz)	Resume and JD text extraction
Animations	Framer Motion	Physics-based UI transitions
Graph View	@xyflow/react	Interactive DAG roadmap visualization
Charts	Recharts	9-axis radar chart for interview results
Persistence (HR)	JSON file (hr_data.json)	Roles and candidates persist across restarts
Persistence (Candidate)	Browser localStorage	Roadmap progress, analysis cache
Deployment (FE)	Vercel	Auto-deploys from GitHub main branch
Deployment (BE)	Render.com	Python web service with env var secrets

3. Dependencies

3.1 Backend Dependencies

Python 3.11+ is required. Install all backend dependencies with:

```
pip install -r requirements.txt
```

Package	Version	Purpose
fastapi	≥0.110.0	REST API framework — all endpoints
uvicorn[standard]	≥0.27.0	ASGI production server
python-multipart	≥0.0.9	Multipart form data (file uploads)
pymupdf	≥1.23.0	PDF text extraction (fitz module)
python-dotenv	≥1.0.0	Load .env environment variables
pydantic	≥2.0.0	Data validation — all request/response models
mangum	≥0.17.0	AWS Lambda handler (optional serverless deploy)

> **No boto3 Required**
> The AWS Bedrock client (ai_client.py) uses Python's built-in urllib for HTTP requests. Authentication uses a long-term Bearer token — no AWS CLI setup, no IAM roles, no SDK.

3.2 Frontend Dependencies

Node 18+ is required. Install all frontend dependencies with:

```
cd frontend && npm install
```

Package	Purpose
react@18, react-dom@18	Core UI framework
react-router-dom@6	SPA routing — 9 routes

Package	Purpose
vite@5	Build tool and dev server
tailwindcss@3	Utility-first CSS framework
framer-motion	Animation library (physics spring, stagger, page transitions)
@xyflow/react	Interactive node graph for DAG roadmap view
recharts	RadarChart for 9-dimension interview results
react-hot-toast	Toast notifications
axios	HTTP client for all API calls
react-dropzone	PDF drag-and-drop upload zones
highlight.js	Code syntax highlighting in interview questions

4. Setup Instructions

4.1 Prerequisites

- Python 3.11 or higher
- Node.js 18 or higher
- An AWS Bedrock long-term API key (get from AWS Console → Bedrock → API keys)
- Git

4.2 Clone the Repository

```
git clone https://github.com/balapraharsha/Gaplytics.git
cd Gaplytics
```

4.3 Backend Setup

1. Navigate to the backend directory:

```
cd backend
```

2. Create your environment file from the template:

```
cp .env.example .env
```

3. Open .env and add your AWS Bedrock API key:

```
AWS_BEARER_TOKEN_BEDROCK=ABSKQm...your_key_here...
AWS_REGION=us-east-1
BEDROCK_MODEL=anthropic.claude-3-haiku-20240307-v1:0
FRONTEND_URL=http://localhost:5173
SEMANTIC_THRESHOLD=0.75
```

4. Install Python dependencies:

```
pip install -r requirements.txt
```

5. Start the development server:

```
uvicorn main:app --reload --port 8000
```

6. Verify it is running:

```
curl http://localhost:8000/api/health
```

Expected response:

```
{"status":"ok","version":"5.0.0","ai":"AWS Bedrock (Claude 3
Haiku)","configured":true}
```

4.4 Frontend Setup

7. Navigate to the frontend directory (from repo root):

```
cd frontend
```

8. Create your environment file:

```
cp .env.example .env
```

9. The default .env points to the local backend — no changes needed for local development:

```
VITE_API_URL=http://localhost:8000
```

10. Install Node dependencies:

```
npm install
```

11. Start the development server:

```
npm run dev
```

12. Open your browser at:

```
http://localhost:5173
```

4.5 Getting Your AWS Bedrock API Key

13. Sign in to the AWS Management Console at console.aws.amazon.com
14. Navigate to Amazon Bedrock in the services search bar
15. In the left sidebar, click API keys
16. Click Generate long-term API key
17. Copy the key (it starts with ABSKQm...) and paste it into backend/.env
18. Ensure model access is enabled: AWS Console → Bedrock → Model access → Enable Claude 3 Haiku

> **Fallback Chain**
> If the primary model (Claude 3 Haiku) returns a 403 or 404, the system automatically tries: Claude 3.5 Haiku → Amazon Titan Text Lite. This ensures the app never hard-crashes due to a single model being unavailable in your AWS region.

4.6 Docker Deployment (Backend)

```
cd backend
docker build -t gaplytics-backend .
docker run -p 8000:8000 --env-file .env gaplytics-backend
```

4.7 Production Deployment

Frontend → Vercel

19. Push your repo to GitHub
20. Go to vercel.com → New Project → Import from GitHub
21. Set Root Directory: frontend
22. Add Environment Variable: VITE_API_URL = your Render backend URL
23. Click Deploy — Vercel auto-deploys on every push to main

Backend →Render

24. Go to render.com →New →Web Service
25. Connect your GitHub repo
26. Set Root Directory: backend
27. Build Command: pip install -r requirements.txt
28. Start Command: uvicorn main:app --host 0.0.0.0 --port $PORT
29. Add all environment variables from .env.example
30. Click Deploy

5. Skill-Gap Analysis: Logic Overview

The skill-gap engine is the analytical core of Gaplytics. It transforms two unstructured text inputs — a resume PDF and a job description — into a precise numerical gap score, a prioritised list of missing skills, and a weighted overall match percentage. The logic is implemented entirely in parser.py, skill_gap.py, and adaptive.py.

5.1 Stage 1 — Text Extraction

Both the resume and the job description enter the pipeline as raw text extracted by PyMuPDF:

```
# parser.py
def extract_text_from_pdf(file_bytes: bytes) -> str:
    doc = fitz.open(stream=file_bytes, filetype='pdf')
    return '\n'.join(page.get_text() for page in doc)
```

The extracted text is capped at 8,000 characters before being sent to the AI model. This keeps token costs low while retaining all meaningful content from typical resume and JD lengths.

5.2 Stage 2 — AI Skill Extraction

The raw text is sent to AWS Bedrock (Claude 3 Haiku) with two separate structured prompts — one for the resume and one for the JD. Each returns a strict JSON array:

```
# Resume extraction returns:
[{
  'name': 'python',
  'category': 'programming_language',
  'proficiency_level': 7,   // 1-10 scale
  'years_experience': 3.0,
  'evidence': 'Built REST APIs with FastAPI and Python 3.11'
}]

# JD extraction returns:
[{
  'name': 'python',
  'category': 'programming_language',
  'required_level': 9,      // 1-10 scale
  'is_mandatory': true,
  'context': 'Strong Python 3 skills required',
  'job_market_relevance_pct': 82
}]
```

Both arrays are validated against Pydantic v2 models (ResumeSkill and JDRequirement) before any matching occurs. Malformed items are silently skipped with a warning log.

5.3 Stage 3 — Skill Name Normalisation

Before any comparison, all skill names pass through normalize_skill_name() which lowercases, strips whitespace, and resolves a hand-curated alias map:

```
ALIAS = {
  'js':        'javascript',
  'ts':        'typescript',
  'py':        'python',
  'k8s':       'kubernetes',
  'ml':        'machine learning',
  'ci/cd':     'ci cd',
  'aws':       'amazon web services',
  'node':      'node.js',
  'reactjs':   'react',
  'springboot':'spring boot',
  # ...20 more aliases
}
```

This ensures that a resume listing 'py' and a JD requiring 'Python' are correctly matched without any AI involvement for this step.

5.4 Stage 4 — Three-Stage Matching

For each JD requirement, the engine attempts to find the best matching skill in the candidate's resume using a three-stage cascade:

Stage	Method	Threshold	Example Match
1 — Exact / Alias	normalize_skill_name() equality	score = 1.0	js → javascript ✓
2 — Substring	One name contains the other	score = 0.85	python 3 → python ✓
3 — Word Overlap	Shared word ratio ≥ 50%	score ≥ 0.50	machine learning eng → machine learning ✓

If no match is found across all three stages, the skill is treated as fully absent (candidate_level = 0). The semantic threshold is configurable via the SEMANTIC_THRESHOLD environment variable.

5.5 Stage 5 — Gap Score Computation

Once a match is found (or not), the gap score is computed as a simple integer difference:

```
gap_score = required_level - candidate_level   # Both on a 1-10 scale

# Priority classification:
if is_mandatory and gap_score >= 5:  priority = 'critical'
elif gap_score >= 3:                 priority = 'high'
elif gap_score >= 1:                 priority = 'medium'
else:                                priority = 'low'
```

Critical gaps appear first in the candidate dashboard and drive the rejection reason predictor. The top three critical/high gaps are surfaced as 'likely rejection factors' with job_market_relevance_pct percentages sourced from the O*NET 30.2 database.

5.6 Stage 6 — Weighted Overall Match Score

The final match percentage uses a mandatory-2x weighting to reflect real-world hiring: a candidate who is missing a mandatory skill should score significantly lower than one who is missing only optional skills.

```
total_weight   = sum(2 if r.is_mandatory else 1 for r in jd_requirements)
matched_weight = 0

for req in jd_requirements:
    candidate_level = find_candidate_level(req, resume_skills)
    match_pct = min(100, (candidate_level / req.required_level) * 100)
    weight = 2 if req.is_mandatory else 1
    matched_weight += (match_pct / 100) * weight

overall_match = round((matched_weight / total_weight) * 100, 1)
```

Score Gate Logic

overall_match_percentage >= 90 →score_gate = 'mock_interview' (interview unlocks)
overall_match_percentage < 90 →score_gate = 'feedback' (InteractiveFeedback shown)

The mock interview route (/candidate/interview) immediately redirects to the dashboard if the stored analysis score is below 90%. This gate is enforced at the route level in AuthorizedInterview.jsx — it is never just hidden, it is genuinely absent from the DOM.

5.7 Stage 7 — Adaptive Learning Pathway

The gap result feeds directly into the adaptive learning pathway generator (adaptive.py), which runs four original algorithms in sequence:

Algorithm	Description
1. Graph-Based Prerequisite DAG	Builds a directed acyclic graph from the 50+ module catalog. Each CourseModule has a prerequisites[] array. Transitive prerequisites are recursively expanded — if you need FastAPI Advanced, Python Intermediate is automatically included.
2. Kahn's Topological Sort (BFS)	Processes the DAG using in-degree tracking. Detects cycles with len(sorted) < len(graph). Produces a valid linear module order from foundational to advanced skills.
3. Deadline-Aware Bin-Packing Scheduler	Packs modules into day slots respecting the daily_hours budget. If required_daily > budget, the system auto-adjusts (×1.1 ceiling, rounded to nearest 0.5h) and shows a warning toast. The last two days are always reserved: final_review then mock_test.
4. Priority-Weighted Gap Scoring	Critical path modules — those addressing mandatory skills — are scheduled in the first 60% of days. Review and rest days fill the remaining slots on a repeating pattern.

6. API Reference

The backend exposes a RESTful API on port 8000. Full OpenAPI documentation is auto-generated at /docs (Swagger UI) and /redoc.

Method	Endpoint	Description
GET	/api/health	Health check — returns version, model, and configured status
GET	/api/ping	Live AI ping — calls Bedrock and returns GAPLYTICS_OK
POST	/api/analyze	Core analysis — resume PDF + JD → full gap result + pathway
POST	/api/chat	AI coach — context-aware response from Claude
POST	/api/interview/generate	Generate 12 interview questions across 6 types
POST	/api/interview/evaluate	Evaluate answers across 9 dimensions
POST	/api/interview/feedback	Get feedback on a single practice answer
POST	/api/hr/create-role	Create HR role from JD — returns role_id + candidate URL
GET	/api/hr/roles	List all HR roles
GET	/api/hr/role/{role_id}	Role detail + full candidate pipeline
GET	/api/hr/candidates/{role_id}	Candidate summaries for a role
POST	/api/hr/compare	AI side-by-side candidate comparison

7. Datasets & Citations

- O*NET 30.2 Database — Powers the ONET_RELEVANCE dict across all 50 course modules. Source: https://www.onetcenter.org/db_releases.html

- Kaggle Resume Dataset — Parser validation and test fixtures. Source: https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset
- Kaggle Jobs & JD Dataset — JD extraction validation across 5 domains. Source: https://www.kaggle.com/datasets/kshitizregmi/jobs-and-job-description

8. Troubleshooting

Error	Cause	Fix
AWS_BEARER_TOKEN_BEDROCK not set	Missing .env key	Add your token to backend/.env
No skills found in resume	Scanned/image PDF	Use a text-based PDF. PyMuPDF cannot OCR images.
CORS error in browser	Backend not running	Ensure uvicorn is on port 8000 and VITE_API_URL matches
JSON parse failed after repair	LLM response truncated	Increase max_tokens in the relevant ai_client.py call
Deadline must be future date	Date validation failed	Set deadline at least 1 day after today (backend enforces this)
Model 403 on Bedrock	Model access not enabled	AWS Console → Bedrock → Model access → Enable Claude 3 Haiku

9. License & Acknowledgements

This project was built for the Gaplytics Hackathon. All AI analysis is powered by AWS Bedrock (Claude 3 Haiku by Anthropic). Skill taxonomy data sourced from O*NET 30.2 (U.S. Department of Labor). Resume and JD test data from Kaggle public datasets.

Repository: https://github.com/balapraharsha/Gaplytics

Live App: https://gaplytics.vercel.app